

## OOP Final Project Reflection Report

B123040023 戴維佑

B123040025 王勁詠

B123040026 張韶軒

### 1. What OOP concept do we use?

#### (1) 封裝 (Encapsulation)

**GridMap 類別：** 將地圖資料 (`self.grid`)、目標位置 (`self.target_pos`) 以及複雜的生成邏輯 (例如 BFS 演算法 `_is_reachable`、隨機生成 `_generate_random_layout`) 都包裝在類別內部。外部使用者 (如 `RobotEnvironment`) 不需要知道地圖是如何生成或 BFS 的細節，只要呼叫 `map.regenerate_map()` 或 `map.get_local_observation()` 就能取得結果。

**QTrainer 類別：** 將 Q-Learning 的演算法細節 (`Alpha`、`Gamma`、`Epsilon` 更新公式) 封裝起來。外部程式 (如 `main.py`) 只需要呼叫 `train()` 或 `test()`，完全不必理解背後的數學公式。

#### (2) 繼承 (Inheritance)

程式碼中的應用：

**GridObject 體系：** `Obstacle`、`Conveyor`、`Target`、`Empty` 都繼承自 `GridObject`。好處是它們自動擁有 `render` 方法 (基礎繪圖能力) 和 `type_id` 屬性，不需要在每個子類別重寫。

**RobotEnvironment 體系：** `RobotEnvironment` 同時繼承自 `WarehouseRobot` 和 `gym.Env`。

- 繼承 `WarehouseRobot`：直接獲得 `_init_pygame` (開啟視窗)、`_load_sprite` (載入圖片)、`render_base` (畫圖) 的能力，環境本身不用重寫繪圖程式碼。
- 繼承 `gym.Env`：讓環境符合 OpenAI Gym 的標準介面，可以被標準的強化學習演算法調用。

#### (3) 多型 (Polymorphism)

程式碼中的應用：

在 `robot_env.py` 的 `step()` 方法中，有一行關鍵程式碼：

```
result = target_obj.interact()
```

這就是多型的應用：

- 如果 `target_obj` 是 `Obstacle` → 執行自己的 `interact`，回傳 `BLOCKED`。
- 如果是 `Conveyor` → 執行自己的 `interact`，回傳 `SLIDE` 和方向。
- 如果是 `Target` → 執行自己的 `interact`，回傳 `WIN`。

好處是未來若要新增「陷阱 (Trap)」，只需建立一個繼承自 `GridObject` 的新類別並實作 `interact`，完全不需要修改 `robot_env.py` 的邏輯。

#### (4) 組合 (Composition)

程式碼中的應用：

**RobotEnvironment 擁有 GridMap：** 在環境類別中有 `self.map = GridMap(...)`。環境本身不是地圖，但它包含地圖物件來協助處理資料。

**GridMap 擁有 GridObject：** `self.grid` 是一個二維陣列，裡面放著各種 `GridObject` 的實例。

#### (5) 抽象 (Abstraction)

程式碼中的應用：

**GridObject 作為抽象基底：** 雖然 Python 沒有強制介面宣告，但在 `GridObject` 中定義了 `interact`，並回傳預設值或拋出錯誤，這就形成一種「契約」。它告訴所有子類別：必須具備 `interact` 的能力，這樣環境才能與它們互動。

## 2.Explain your implementation in detail

在 `part2` 中，我們嘗試調了 `learning rate`、`epsilon_decay_rate` 等參數，還有新增掉進洞裡給予懲罰(負獎勵)的機制，讓他訓練時可以知道不要這樣走，`epsilon_decay_rate` 設為 `0.99965` 可以使她在前中期保留較多隨機性，而後期的時候比較不會隨機亂走，但是我們最後只有成功調到穩定 `60%`。

`Part3` 的部分，我們首先將機器人加入了斜走的功能，之後再往地圖物件的方向

發展，加入了傳送帶、障礙物等等，並且利用多形的概念讓程式的結構變得相對簡短。

### 3.What you learned from this assignment?

我們學到了如何運用 OOP 的概念改良一款遊戲，還有畫出 UML 圖可以更方便的和團隊中的其他人溝通，並且使用 UML 圖分工可以避免功能重疊或邏輯衝突，這在大型專案中是很重要的能力。

### 4. Do you use AI tool in this assignment?

### 5.How you used a AI tool assignment?

### 6. Evaluation of the tool's usefulness?

我們首先使用 AI 幫助我們釐清題目的要求，然後嘗試自己調參數和寫程式，在遇到無法達成條件時，如 part2 無法達到 70%，就會詢問 AI 是否有更好的解答，但我們的結果依然沒有變得更好，所以 AI 可以用來幫助 debug 或是解釋題目，但是如果要直接尋求答案，可能要經過長時間的訓練才比較可能有滿意的答案。